

BCSE355L – Cloud Architecture Design

Project Phase-I

InfraDriftGuard

InfraDriftGuard: AI-Based Infrastructure Configuration Drift Detection and Risk
Classification on AWS using Explainable Machine Learning

Submitted by

Keshav Khandelwal — 24BIT0461

Somasundar Shivappa Badami — 24BIT0465

Course Instructor: Dr. Priya V

GitHub: [InfraDriftGuard_Cloud_Project_2026](#)

Contents

Right-click here and choose “Update Field” to build the table of contents.

Abstract

Infrastructure as Code (IaC) tools such as Terraform declare cloud infrastructure in version-controlled files, but the live environment rarely stays identical to that declaration. Manual console edits, emergency fixes and automation side-effects cause configuration drift, and drift in security-sensitive settings – a public S3 bucket, SSH opened to the internet, an IAM role escalated to wildcard permissions – is one of the leading causes of cloud data breaches. Existing tools have three shortcomings: static IaC scanners inspect only the code and never see the live account; native drift checks (terraform plan, AWS Config) report that something changed but not how dangerous the change is; and the few machine-learning approaches published so far are black boxes that operators cannot audit. This project proposes InfraDriftGuard, a serverless drift detection and risk classification system on AWS. Terraform renders the desired state of every monitored resource; an AWS Lambda function, triggered hourly by Amazon EventBridge, collects the live configuration of S3 buckets, EC2 security groups and IAM roles through read-only APIs, normalises it, computes an attribute-level diff, and converts each change into nine security features. A Random Forest classifier, trained on a 711-record dataset covering 20 drift scenarios and exported to a dependency-free JSON model, assigns a Low/Medium/High/Critical risk, while SHAP explains which features drove each prediction. Changes the model was never trained on are labelled “Review” instead of guessed. Findings are stored in Amazon DynamoDB, High/Critical drift triggers Amazon SNS e-mail alerts, two unambiguous Critical patterns can be auto-remediated, and results are served through Amazon API Gateway secured by Amazon Cognito to a Streamlit dashboard. Amazon CloudWatch provides logs, metrics, alarms and a dashboard, and AWS Budgets guards the free-tier cost. AWS services used: Lambda, EventBridge, S3, EC2, IAM, DynamoDB, SNS, API Gateway, Cognito, CloudWatch and Budgets.

Keywords: Infrastructure as Code, configuration drift, AWS, Terraform, serverless, Random Forest, SHAP, explainable AI, cloud security

Introduction

Problem statement. Cloud teams declare infrastructure in Terraform, but the live AWS account drifts away from that declaration through console edits, hot-fixes and automation. Drift in security settings – public S3 buckets, internet-facing admin ports, over-privileged IAM roles – is a leading cause of cloud breaches, yet today's tools either never look at the live account or report every drift with the same urgency and no explanation.

Existing challenges. Static scanners miss post-deployment changes; terraform plan and AWS Config show differences without severity; ML and LLM approaches are opaque, costly or hallucinate; automated responses are rare and unsafe.

Proposed solution. InfraDriftGuard continuously compares Terraform's desired state with live AWS configuration, classifies every drift into four risk levels with an explainable Random Forest, and responds through a serverless AWS pipeline (alerts, storage, optional safe remediation, secured API and dashboard).

Literature Survey

Fifteen peer-reviewed papers published 2023–2026 (IEEE, ACM, Springer, Frontiers and Scopus-indexed journals) were studied. Papers 1–8 were studied by Student 1 and papers 9–15 by Student 2, following the team-of-two allocation.

Paper	Method	Dataset	Advantages	Limitations	Research Gap
[1] Thiagarajan et al., IJCNIS 2024	ML models trained on historical and real-time configuration data from AWS Config logs and Terraform state to flag drift; integrates with existing IaC pipelines.	AWS Config configuration-item history and Terraform state from cloud deployments.	Directly targets drift in AWS; uses live configuration, not only code; shows ML can learn which configuration changes matter.	Relies on AWS Config (a paid service per recorded item); model decisions are not explained to the operator; no per-change risk severity or automated response.	Drift is detected but not ranked by security impact, and predictions are not explainable.
[2] Hassan et al., ACM FSE 2024	Empirical study of defects in how IaC tools reconcile declared state with real state; builds a taxonomy and a testing approach for state-reconciliation bugs.	Defect reports and code changes from Ansible, Terraform-style IaC projects and tools.	First systematic look at the gap between declared and actual state; explains why drift arises even without human error.	Focuses on finding bugs in the IaC tools themselves, not on monitoring production accounts or rating the risk of a given drift.	No runtime component that continuously checks real cloud resources against the declared state.
[3] Abuzakuk et al. (RIVA), ACM 2026	Multi-agent LLM system (verifier agent + tool-generation agent) that cross-validates tool outputs to verify that deployed systems match their IaC specification.	IaC verification tasks with injected unreliable/incorrect tool responses.	Robust to misleading tool output; raises task success from 27.3% to 50.0% over a ReAct baseline when tools are unreliable.	Even the best configuration solves only about half of the tasks; LLM inference is costly and non-deterministic; no severity rating of detected drift.	Accurate, cheap and deterministic drift checking with a quantified risk is still missing.
[4] Wen et al. (SIsDetector), ACM TOSEM 2026	SIsDetector: zero-shot prompt-engineered LLM that inspects AWS SAM templates for misconfigurations.	Real-world AWS SAM serverless application configurations.	No training data needed; precision 72.88%, recall 88.18%, F1 79.75%, beating data-driven baselines.	Analyses templates before deployment only; about one in four alarms is a false positive; requires an external LLM API.	Misconfigurations introduced after deployment (outside the template) are invisible to it.
[5] Rahman et al., ACM TOSEM 2023	Qualitative analysis of 2,039 Kubernetes manifests from 92 repositories; builds the SLI-KUBE static linter for 11 misconfiguration categories; logistic regression on code metrics.	2,039 manifests (GitHub 1,590 + GitLab 449) from 92 open-source repositories.	Large real-world dataset; identified 1,051 misconfigurations; practitioners agreed to fix 60% of reported issues. (This is the base paper replicated by our team.)	Static analysis of manifests only; binary present/absent output with no severity; Kubernetes-specific.	Severity of each misconfiguration and its appearance at runtime are not addressed.
[6] Saavedra et al. (GLITCH), IEEE/ACM ASE 2023	Technology-agnostic intermediate representation of IaC scripts with rule-based detectors for 9 security smells and 9 design smells.	IaC scripts in Ansible, Chef, Docker, Puppet and Terraform.	One analyser for five IaC languages; consistent rules across technologies.	Rule-based and static; cannot see changes made outside the code; no learned prioritisation.	No link between code-level smells and the state actually running in the cloud.

Paper	Method	Dataset	Advantages	Limitations	Research Gap
[7] Opdebeeck et al. (GASEL), IEEE/ACM MSR 2023	GASEL: graph queries over program dependence graphs of Ansible code to detect 7 security smells with control- and data-flow awareness.	More than 15,000 Ansible scripts; oracle of 243 real security smells.	Substantially better precision and recall than earlier detectors; shows that 55% of smells involve data-flow indirection.	Ansible-only; static; high analysis cost (32% of smells need whole-project analysis).	Effort is spent on deeper static analysis while runtime drift remains undetected.
[8] Verdet et al., Springer EMSE 2025	Mined 812 open-source Terraform projects and checked them against security policies for AWS, Azure and Google Cloud.	812 GitHub Terraform projects.	Cross-provider evidence of which security practices developers adopt or neglect (e.g. encryption, access control, logging).	Measures what is written in code, not what is deployed; no remediation.	Even code that follows the policies can drift once deployed; this is not measured.
[9] Saavedra et al. (InfraFix), ACM ISSTA 2025	SMT-based repair module plus system-call-based state inference to repair IaC scripts.	254,288 generated repair scenarios.	95.7% repair success; technology-agnostic; considers the observed system state.	Repairs the script, not the running resource; evaluated on synthetic scenarios.	No safe, policy-limited remediation of the live cloud resource itself.
[10] Low, Chen et al., IEEE SecDev 2024	Feeds scanner findings, IaC code and human context to GPT-4 in a two-pass prompt to generate repaired IaC.	Vulnerable open-source IaC repositories scanned with existing misconfiguration scanners.	Removes up to 84.7% of scanner alarms; two-pass prompting clearly beats one-pass.	20.4% of suggested fixes were hallucinated; needs a human in the loop; no risk ranking.	Fixes are not prioritised and are not verified against the deployed state.
[11] Begoug et al., IEEE/ACM MSR 2024	Six ML algorithms (LightGBM best) predict defective Terraform blocks from code, process and change metrics.	19 open-source Terraform projects.	Block-level granularity; shows ML can prioritise risky IaC changes (AUC 0.71).	Low MCC (0.21); predicts defects in commits, not security impact of live changes; limited explainability.	No ML model that predicts the security severity of a configuration change in the live cloud.
[12] Begoug et al. (TerraMetrics), IEEE/ACM ICPC 2024	Parses Terraform HCL into an AST and extracts a catalogue of 40 quality metrics as JSON.	Terraform projects (tool paper; metrics catalogue).	Reusable, open-source feature extractor for Terraform; well-defined metrics.	Quality metrics of code only; nothing about security impact or runtime state.	No equivalent feature catalogue for drift between Terraform and live AWS resources.
[13] Rjoub et al., IEEE TNSM 2023	Survey of XAI techniques (SHAP, LIME, rule extraction, attention) applied to intrusion, malware and fraud detection.	Survey of published XAI-for-security studies.	Argues that security analysts need explanations to trust and act on ML alerts; maps techniques to use cases.	Cloud configuration management is not covered; no implementation.	XAI has not been applied to configuration drift risk on cloud platforms.
[14] Mohale & Obagbuwa, Frontiers in AI 2025	Systematic literature review of XAI methods in intrusion detection systems.	Peer-reviewed IDS + XAI studies (systematic review).	Finds tree-based and rule-based models with SHAP preferred for interpretability; identifies need for real-time explainability.	Network IDS focus; notes trade-off between accuracy and interpretability but proposes no system.	Real-time, explainable tree-based classification in a cloud operations pipeline.

Paper	Method	Dataset	Advantages	Limitations	Research Gap
[15] Saleh et al., ACM CCSW 2024	CNN + LSTM model detects anomalous network traffic in CI/CD pipelines and cloud platforms.	Network traffic data from pipeline and cloud environments.	High accuracy (98.69% and 98.30%) for traffic anomalies in DevOps pipelines.	Deep models are opaque; looks at traffic, not configuration; heavy to deploy serverlessly.	Configuration-level security changes are not monitored; model is not explainable.

Summary of the survey

The fifteen papers fall into four groups. (1) Static IaC analysis – GLITCH, GASEL, SLI-KUBE, TerraMetrics, Verdet et al. and SIsDetector – finds misconfigurations in code before deployment, but cannot see changes made afterwards. (2) Drift and state reconciliation – Thiyagarajan et al., Hassan et al. and RIVA – confirm that drift is real and common, but either do not rate how dangerous a drift is or rely on costly, non-deterministic LLM agents. (3) Automated repair – InfraFix and LLM-based repair – fix scripts, sometimes with hallucinated changes, and do not act on the live resource. (4) ML and explainability for security – Begoug et al., Saleh et al., Rjoub et al. and Mohale & Obagbuwa – show that ML can prioritise risk and that analysts need explanations, yet none of them combines these with live cloud configuration. InfraDriftGuard sits at the intersection: runtime drift detection on real AWS resources, a per-change risk level from an explainable model, and a serverless, low-cost response path.

Research Gap Analysis — Student 1 (Papers 1–8)

Prepared by Keshav Khandelwal (24BIT0461). Gaps are stated in our own words based on our reading of each paper, not copied from the papers.

[1] Thiyagarajan et al., IJCNIS 2024

Existing method: ML models trained on historical and real-time configuration data from AWS Config logs and Terraform state to flag drift; integrates with existing IaC pipelines.

Advantages: Directly targets drift in AWS; uses live configuration, not only code; shows ML can learn which configuration changes matter.

Limitations: Relies on AWS Config (a paid service per recorded item); model decisions are not explained to the operator; no per-change risk severity or automated response.

Research gap: Drift is detected but not ranked by security impact, and predictions are not explainable.

Possible improvement (addressed in InfraDriftGuard): Classify each drift into a severity level and attach SHAP explanations; collect state directly with read-only APIs to avoid AWS Config cost.

[2] Hassan et al., ACM FSE 2024

Existing method: Empirical study of defects in how IaC tools reconcile declared state with real state; builds a taxonomy and a testing approach for state-reconciliation bugs.

Advantages: First systematic look at the gap between declared and actual state; explains why drift arises even without human error.

Limitations: Focuses on finding bugs in the IaC tools themselves, not on monitoring production accounts or rating the risk of a given drift.

Research gap: No runtime component that continuously checks real cloud resources against the declared state.

Possible improvement (addressed in InfraDriftGuard): Run a scheduled detector in the cloud that compares Terraform-rendered desired state with live configuration.

[3] Abuzakuk et al. (RIVA), ACM 2026

Existing method: Multi-agent LLM system (verifier agent + tool-generation agent) that cross-validates tool outputs to verify that deployed systems match their IaC specification.

Advantages: Robust to misleading tool output; raises task success from 27.3% to 50.0% over a ReAct baseline when tools are unreliable.

Limitations: Even the best configuration solves only about half of the tasks; LLM inference is costly and non-deterministic; no severity rating of detected drift.

Research gap: Accurate, cheap and deterministic drift checking with a quantified risk is still missing.

Possible improvement (addressed in InfraDriftGuard): Use a deterministic diff engine for detection and a lightweight, explainable classifier only for risk scoring.

[4] Wen et al. (SIsDetector), ACM TOSEM 2026

Existing method: SIsDetector: zero-shot prompt-engineered LLM that inspects AWS SAM templates for misconfigurations.

Advantages: No training data needed; precision 72.88%, recall 88.18%, F1 79.75%, beating data-driven baselines.

Limitations: Analyses templates before deployment only; about one in four alarms is a false positive; requires an external LLM API.

Research gap: Misconfigurations introduced after deployment (outside the template) are invisible to it.

Possible improvement (addressed in InfraDriftGuard): Monitor the live account after deployment and compare it to the template's intent.

[5] Rahman et al., ACM TOSEM 2023

Existing method: Qualitative analysis of 2,039 Kubernetes manifests from 92 repositories; builds the SLI-KUBE static linter for 11 misconfiguration categories; logistic regression on code metrics.

Advantages: Large real-world dataset; identified 1,051 misconfigurations; practitioners agreed to fix 60% of reported issues. (This is the base paper replicated by our team.)

Limitations: Static analysis of manifests only; binary present/absent output with no severity; Kubernetes-specific.

Research gap: Severity of each misconfiguration and its appearance at runtime are not addressed.

Possible improvement (addressed in InfraDriftGuard): Extend from static manifests to live AWS resources and grade each finding by risk.

[6] Saavedra et al. (GLITCH), IEEE/ACM ASE 2023

Existing method: Technology-agnostic intermediate representation of IaC scripts with rule-based detectors for 9 security smells and 9 design smells.

Advantages: One analyser for five IaC languages; consistent rules across technologies.

Limitations: Rule-based and static; cannot see changes made outside the code; no learned prioritisation.

Research gap: No link between code-level smells and the state actually running in the cloud.

Possible improvement (addressed in InfraDriftGuard): Use a common normalised schema for live resources, analogous to GLITCH's IR for code.

[7] Opdebeeck et al. (GASEL), IEEE/ACM MSR 2023

Existing method: GASEL: graph queries over program dependence graphs of Ansible code to detect 7 security smells with control- and data-flow awareness.

Advantages: Substantially better precision and recall than earlier detectors; shows that 55% of smells involve data-flow indirection.

Limitations: Ansible-only; static; high analysis cost (32% of smells need whole-project analysis).

Research gap: Effort is spent on deeper static analysis while runtime drift remains undetected.

Possible improvement (addressed in InfraDriftGuard): Complement static analysis with cheap runtime state comparison.

[8] Verdet et al., Springer EMSE 2025

Existing method: Mined 812 open-source Terraform projects and checked them against security policies for AWS, Azure and Google Cloud.

Advantages: Cross-provider evidence of which security practices developers adopt or neglect (e.g. encryption, access control, logging).

Limitations: Measures what is written in code, not what is deployed; no remediation.

Research gap: Even code that follows the policies can drift once deployed; this is not measured.

Possible improvement (addressed in InfraDriftGuard): Use the same policy areas (public access, encryption, IAM) as the features of a runtime drift classifier.

Research Gap Analysis — Student 2 (Papers 9–15)

Prepared by Somasundar Shivappa Badami (24BIT0465). Gaps are stated in our own words based on our reading of each paper, not copied from the papers.

[9] Saavedra et al. (InfraFix), ACM ISSTA 2025

Existing method: SMT-based repair module plus system-call-based state inference to repair IaC scripts.

Advantages: 95.7% repair success; technology-agnostic; considers the observed system state.

Limitations: Repairs the script, not the running resource; evaluated on synthetic scenarios.

Research gap: No safe, policy-limited remediation of the live cloud resource itself.

Possible improvement (addressed in InfraDriftGuard): Offer opt-in, narrowly scoped auto-remediation only for unambiguous Critical drift.

[10] Low, Chen et al., IEEE SecDev 2024

Existing method: Feeds scanner findings, IaC code and human context to GPT-4 in a two-pass prompt to generate repaired IaC.

Advantages: Removes up to 84.7% of scanner alarms; two-pass prompting clearly beats one-pass.

Limitations: 20.4% of suggested fixes were hallucinated; needs a human in the loop; no risk ranking.

Research gap: Fixes are not prioritised and are not verified against the deployed state.

Possible improvement (addressed in InfraDriftGuard): Rank findings by predicted risk first and only automate fixes that are deterministic.

[11] Begoug et al., IEEE/ACM MSR 2024

Existing method: Six ML algorithms (LightGBM best) predict defective Terraform blocks from code, process and change metrics.

Advantages: Block-level granularity; shows ML can prioritise risky IaC changes (AUC 0.71).

Limitations: Low MCC (0.21); predicts defects in commits, not security impact of live changes; limited explainability.

Research gap: No ML model that predicts the security severity of a configuration change in the live cloud.

Possible improvement (addressed in InfraDriftGuard): Train a classifier on security-oriented features of each drift and explain it.

[12] Begoug et al. (TerraMetrics), IEEE/ACM ICPC 2024

Existing method: Parses Terraform HCL into an AST and extracts a catalogue of 40 quality metrics as JSON.

Advantages: Reusable, open-source feature extractor for Terraform; well-defined metrics.

Limitations: Quality metrics of code only; nothing about security impact or runtime state.

Research gap: No equivalent feature catalogue for drift between Terraform and live AWS resources.

Possible improvement (addressed in InfraDriftGuard): Define a fixed, documented feature schema for drift (our nine features).

[13] Rjoub et al., IEEE TNSM 2023

Existing method: Survey of XAI techniques (SHAP, LIME, rule extraction, attention) applied to intrusion, malware and fraud detection.

Advantages: Argues that security analysts need explanations to trust and act on ML alerts; maps techniques to use cases.

Limitations: Cloud configuration management is not covered; no implementation.

Research gap: XAI has not been applied to configuration drift risk on cloud platforms.

Possible improvement (addressed in InfraDriftGuard): Attach SHAP explanations to every drift risk prediction.

[14] Mohale & Obagbuwa, Frontiers in AI 2025

Existing method: Systematic literature review of XAI methods in intrusion detection systems.

Advantages: Finds tree-based and rule-based models with SHAP preferred for interpretability; identifies need for real-time explainability.

Limitations: Network IDS focus; notes trade-off between accuracy and interpretability but proposes no system.

Research gap: Real-time, explainable tree-based classification in a cloud operations pipeline.

Possible improvement (addressed in InfraDriftGuard): Use a Random Forest + TreeSHAP and run inference inside the detection Lambda.

[15] Saleh et al., ACM CCSW 2024

Existing method: CNN + LSTM model detects anomalous network traffic in CI/CD pipelines and cloud platforms.

Advantages: High accuracy (98.69% and 98.30%) for traffic anomalies in DevOps pipelines.

Limitations: Deep models are opaque; looks at traffic, not configuration; heavy to deploy serverlessly.

Research gap: Configuration-level security changes are not monitored; model is not explainable.

Possible improvement (addressed in InfraDriftGuard): Watch configuration rather than traffic, with a light, explainable model.

Consolidated Research Gap

Taken together, the surveyed work leaves the following gaps, which define the scope of this project:

- Static scanners never observe the deployed account, so post-deployment drift goes unnoticed.
- Drift detectors report that something changed, but not how severe it is, so every alert looks equally urgent.
- ML-based approaches are opaque; none explains which property of the change made it risky.
- LLM-based approaches are expensive, non-deterministic and hallucinate a measurable share of their output.
- Automated responses either do not exist or rewrite code; safe, scoped remediation of the live resource is missing.
- No published system is designed to run within free-tier/serverless cost limits for small teams and students.

Project Objectives

1. **Detect drift continuously:** Compare the Terraform-declared desired state with the live configuration of S3 buckets, EC2 security groups and IAM roles every hour (and on demand), detecting 100% of the 21 supported attribute changes in the automated test suite.
2. **Classify risk accurately:** Assign each drift a Low/Medium/High/Critical risk with a Random Forest classifier, reaching at least 95% macro-F1 on held-out records, and report the harder scenario-held-out score honestly alongside it.
3. **Explain every prediction:** Provide SHAP global and local explanations whose values add up exactly to the model output (additivity error below $1e-6$) so an operator can see why a change was rated Critical.
4. **Respond automatically and safely:** Send an SNS e-mail for every High/Critical drift within one scan cycle, store every finding in DynamoDB, and optionally auto-revert the two Critical patterns that have a single safe fix.
5. **Secure access and observability:** Expose findings only through an API Gateway endpoint protected by Cognito JWT authentication, and monitor the pipeline with CloudWatch logs, metrics, alarms and a dashboard.
6. **Stay inside the AWS free tier:** Keep the full deployment under US\$1 per month (enforced by an AWS Budgets alarm) by running inference in a dependency-free Lambda package of under 1 MB.

Novelty Summary

What makes InfraDriftGuard different from existing work:

- **Better architecture – runtime, not just code.** Unlike static scanners (GLITCH, GASEL, SLI-KUBE, SIsDetector), InfraDriftGuard checks the live AWS account against the Terraform-rendered desired state, catching console edits and emergency changes made after deployment.
- **New feature – risk level for every drift.** Instead of a binary “drift / no drift” (terraform plan, AWS Config, Thiagarajan et al.), each change is converted into nine security features (public exposure, encryption change, privilege change, port exposure, sensitivity, magnitude, frequency, resource type, attribute) and classified into four risk levels.
- **Better explainability.** SHAP TreeExplainer gives exact (additivity error $\approx 1e-16$) global and per-finding explanations, answering the gap raised by the XAI surveys of Rjoub et al. and Mohale & Obagbuwa.
- **Better reliability – no guessing.** Changes outside the model's knowledge base are labelled “Review” and escalated, rather than forced into a class – a direct response to the hallucination and false-positive problems of LLM-based approaches.
- **Better AWS integration and automation.** A fully serverless pipeline – EventBridge → Lambda → DynamoDB / SNS / CloudWatch, with API Gateway + Cognito for access – deployed by one Terraform command, with opt-in auto-remediation limited to two deterministic fixes (re-enable S3 public-access block; revoke 0.0.0.0/0 on SSH/RDP).
- **Better cost and scalability.** The 200-tree Random Forest is exported to a 210 KB JSON file and evaluated in pure Python, giving bit-identical predictions to scikit-learn on all 711 records with no ML libraries in Lambda. The whole stack runs within the free tier and scales by simply adding resources to the desired-state list.
- **Better security of the tool itself.** Each Lambda has its own least-privilege IAM role; collection is read-only; write permissions for remediation exist only when the feature is switched on and are scoped to the specific resources.

Proposed Architecture

Diagram 1 — AWS Cloud Architecture

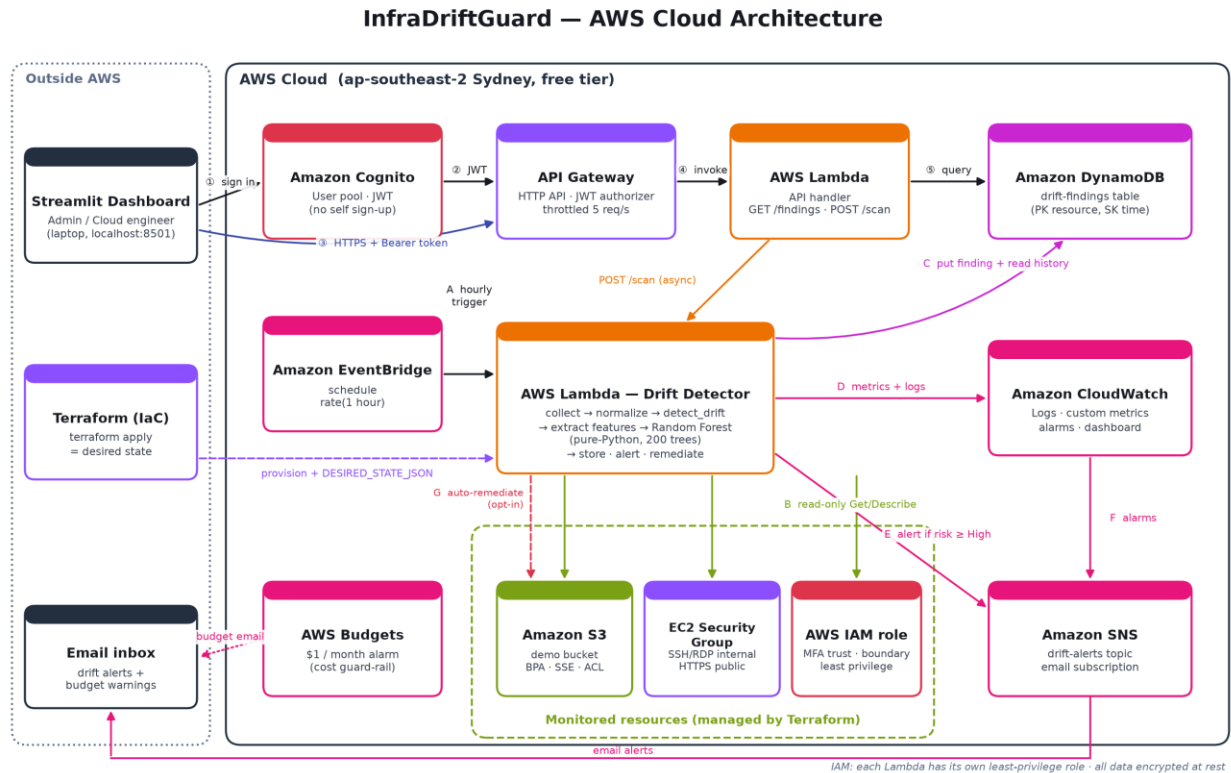


Figure 1. AWS cloud architecture of InfraDriftGuard.

Aspect	How it is realised
Data flow	EventBridge → Detector Lambda → (boto3 read-only) S3 / EC2 SG / IAM → Detector → DynamoDB, CloudWatch, SNS. Dashboard → Cognito → API Gateway → API Lambda → DynamoDB.
Storage	DynamoDB table drift-findings (partition key resource_id, sort key detected_at), encrypted at rest; model shipped inside the Lambda package.
Processing	Detector Lambda: collect → normalise → detect_drift → extract 9 features → pure-Python Random Forest → risk label; API Lambda for queries and on-demand scans.
Authentication	Amazon Cognito user pool (admin-created users, strong password policy, 1-hour tokens); API Gateway JWT authorizer on every route; IAM least-privilege roles.
Notifications	Amazon SNS e-mail for High/Critical/Review drift and for CloudWatch alarms; AWS Budgets e-mails for cost.
Monitoring	CloudWatch Logs (7-day retention), custom metric InfraDriftGuard/DriftDetected by RiskLevel, alarms on Lambda errors and Critical drift, CloudWatch dashboard.

Diagram 2 — Complete System Architecture

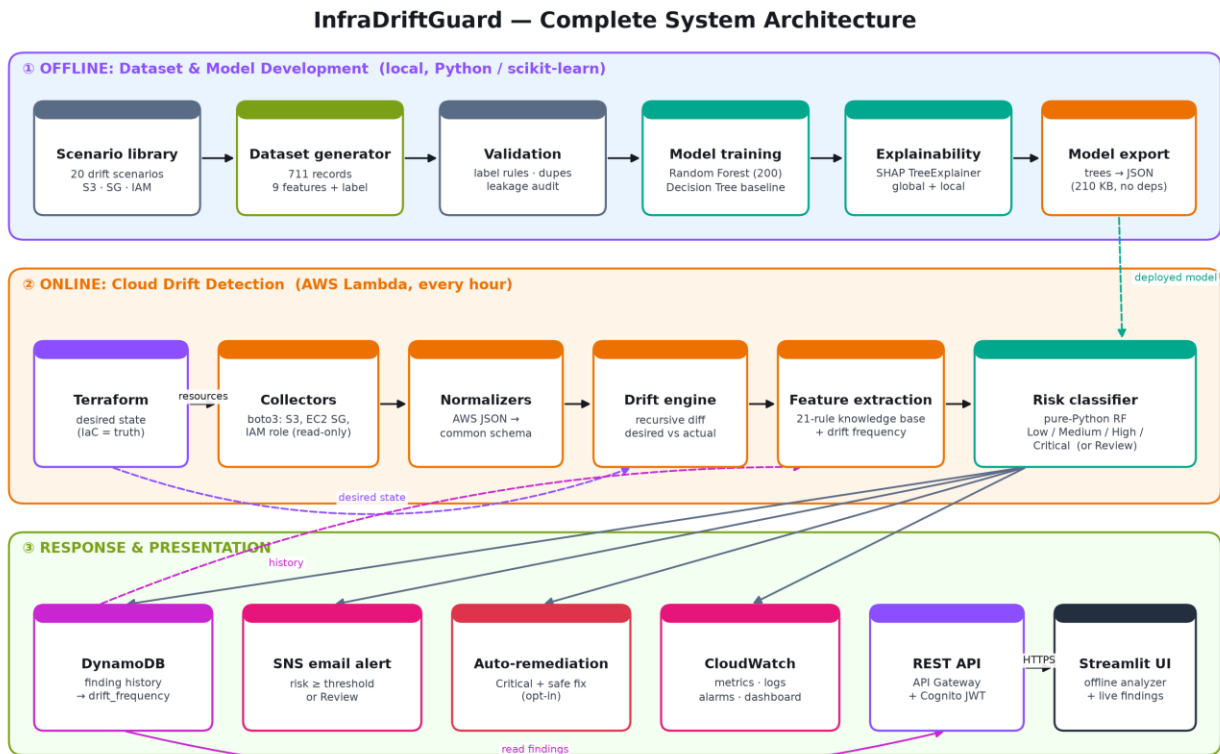


Figure 2. Complete system architecture: offline model development, online detection, response and presentation.

The system has three layers. The offline layer builds the scenario library, generates and validates the dataset, trains the Random Forest and Decision Tree, produces SHAP explanations and exports the model to JSON. The online layer runs in AWS Lambda: Terraform supplies the desired state, collectors read live configuration, normalisers map it to a common schema, the drift engine diffs the two and the feature extractor and classifier rate the risk. The response and presentation layer stores findings, alerts, optionally remediates, publishes metrics and serves the results to users.

Scan workflow

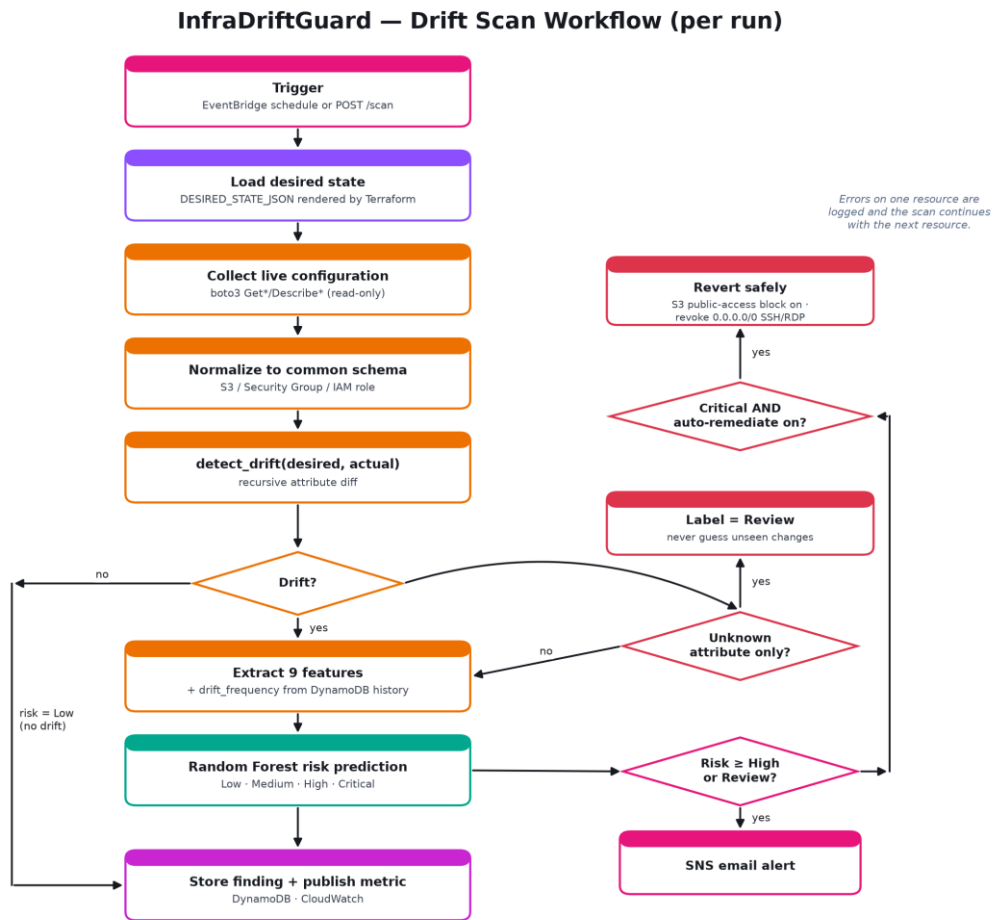


Figure 3. Decision flow of a single drift scan.

Dataset Details

Field	Details
Dataset name	InfraDriftGuard Synthetic Cloud Configuration Drift Dataset (v1.0)
Source	Generated by this project (src/backend/dataset_generator.py) from 20 expert-defined drift scenarios modelled on AWS security best practices and CIS AWS Foundations Benchmark controls.
URL	GitHub repository: dataset/processed/dataset.csv and dataset/raw/raw_records.json (InfraDriftGuard_Cloud_Project_2026)
Size	dataset.csv ≈ 68 KB; raw_records.json ≈ 1.3 MB
Number of records	711 records (Low 180, Medium 176, High 175, Critical 180)
Number of features	9 input features + 1 label (plus 3 metadata columns: record_id, scenario_id, group_id)
Data type	Categorical (resource_type, changed_attribute, security_sensitivity, change_magnitude, port_exposure), Boolean (public_exposure, encryption_change, privilege_change), Integer (drift_frequency); label: ordinal categorical (Low < Medium < High < Critical). Raw file: nested JSON desired/actual states.
License	MIT License (same as the repository); contains no personal or customer data.
Purpose of using the dataset	Train and evaluate the drift risk classifier and SHAP explanations. Public datasets of labelled configuration drift do not exist (real drift data is private to each organisation), so a controlled synthetic dataset with documented ground-truth rules is used, and the live AWS pipeline is validated separately.

Features

Feature	Type	Meaning
resource_type	Categorical	s3_bucket / security_group / iam_policy
changed_attribute	Categorical	Which attribute(s) drifted, e.g. block_public_access
security_sensitivity	Ordinal	Intrinsic sensitivity of the attribute (Low/Medium/High)
public_exposure	Boolean	Did the change expose the resource to the internet?
encryption_change	Boolean	Was encryption weakened or removed?
privilege_change	Boolean	Were permissions escalated or a control removed?
port_exposure	Ordinal	none / internal / 0.0.0.0/0
change_magnitude	Ordinal	None/Low/Medium/High size of the change
drift_frequency	Integer	How often this resource has drifted before (from DynamoDB history)
risk_label (target)	Ordinal	Low / Medium / High / Critical

Drift scenarios

ID	Resource	Drift	Risk
S1	S3	Public access block disabled	Critical
S2	S3	Bucket ACL changed to public-read	Critical
S3id	S3	Server-side encryption disabled	High

ID	Resource	Drift	Risk
S4	S3	Bucket policy principal widened to *	Critical
S5	S3	Versioning disabled	Medium
S6	S3	Access logging disabled	Medium
S7	S3	Lifecycle/retention rule removed	Low
E1	Security group	SSH (22) opened to 0.0.0.0/0	Critical
E2	Security group	RDP (3389) opened to 0.0.0.0/0	Critical
E3	Security group	New port opened publicly	High
E4	Security group	Outbound widened to allow-all	High
E5	Security group	No change (control)	Low
E6	Security group	Internal port widened to a range	Medium
E7	Security group	Cross-SG reference added	Medium
I1	IAM	Policy escalated to Action:* Resource:*	Critical
I2	IAM	Cross-account trust added	Critical
I3	IAM	MFA condition removed	High
I4	IAM	Explicit Deny flipped to Allow	Critical
I5	IAM	Group membership escalated to admin	Critical
I6	IAM	Permissions boundary removed	High

Preprocessing required

- Read with keep_default_na=False so the literal category “None” (no-drift control) is not turned into a missing value.
- Drop metadata columns (record_id, scenario_id, group_id) to prevent the model memorising scenarios.
- Ordinal-encode security_sensitivity, change_magnitude and port_exposure in severity order.
- One-hot encode resource_type and changed_attribute (unknown categories ignored at inference).
- Cast Booleans to 0/1; pass drift_frequency through unchanged.
- Encode the label with a fixed severity order (Low=0 ... Critical=3).
- Two splits: stratified random 80/20 (568/143) and scenario-group-aware (592/119) to measure leakage.

AWS Services Planning

AWS Service	Purpose in InfraDriftGuard	Free-tier position
AWS Lambda	Runs the drift detector (collect, diff, features, Random Forest, alert, remediate) and the REST API handler.	1M requests + 400,000 GB-s / month (always free)
Amazon EventBridge	Hourly schedule that triggers the detector.	Scheduled rules free
Amazon S3	Monitored demo bucket (public-access block, encryption, ACL, versioning).	Empty bucket ≈ \$0
Amazon EC2 (Security Groups)	Monitored firewall rules (SSH/RDP/HTTPS/PostgreSQL).	Security groups are free
AWS IAM	Monitored application role; least-privilege roles for both Lambdas; permissions boundary.	Free
Amazon DynamoDB	Stores every finding; history gives the drift_frequency feature.	25 GB + 25 RCU/WCU (always free); table uses 1/1
Amazon SNS	E-mail alerts for High/Critical/Review drift and CloudWatch alarms.	1,000 e-mails / month free
Amazon API Gateway (HTTP API)	Authenticated REST endpoints: GET /findings, GET /findings/{id}, POST /scan; throttled.	≈ \$1 per million requests; demo usage ≈ \$0
Amazon Cognito	User pool and JWT tokens for dashboard sign-in; admin-created users only.	10,000 MAU free (Lite tier)
Amazon CloudWatch	Lambda logs (7-day retention), custom DriftDetected metric, 2 alarms, dashboard.	10 alarms, 3 dashboards, 5 GB logs free
AWS Budgets	US\$1 monthly budget with e-mail alerts at 50% actual / 100% forecast.	First 2 budgets free
Terraform (IaC, not an AWS service)	Provisions all of the above and renders the desired state.	Free, open source

Note: AWS accounts created after 15 July 2025 use the credit-based Free Plan (up to US\$200 in credits for six months) in addition to the always-free allowances listed above. The \$1 AWS Budget alarm e-mails the team long before any charge could matter.

Implementation

Repository modules

Module	Responsibility
src/backend/drift_engine.py	Generic recursive diff of desired vs actual state
src/backend/feature_extraction.py	21-rule knowledge base converting each change to 9 features
src/backend/scenario_definitions.py, dataset_generator.py	20 scenarios → 711-record dataset
src/ml_model/ml_pipeline.py	Preprocessing, two splits, Random Forest + Decision Tree, metrics
src/ml_model/shap_explainability.py	SHAP TreeExplainer, global/local plots, additivity check

Module	Responsibility
src/ml_model/export_model.py	Exports the Random Forest + encoder to dependency-free JSON
src/aws/collectors/*	boto3 collection and normalisation for S3, security groups, IAM roles
src/aws/lambda/detector_handler.py	Scheduled detector: diff, classify, store, alert, remediate
src/aws/lambda/api_handler.py	REST API behind API Gateway + Cognito
src/aws/lambda/rf_predict.py	Pure-Python Random Forest inference
src/aws/terraform/*.tf	All AWS infrastructure + rendered desired state
src/frontend/app.py, cloud_client.py	Streamlit dashboard: offline analyzer + live AWS findings
tests/	219 automated tests (pytest + moto simulated AWS)

Key design decisions

- **Terraform as single source of truth:** the desired state the Lambda compares against is rendered by Terraform from the same resources it creates, so code and monitor cannot disagree.
- **Dependency-free model:** 200 decision trees are serialised (5,472 nodes, 210 KB) and evaluated in pure Python; a parity test confirms identical probabilities to scikit-learn for all 711 records.
- **Real drift_frequency:** computed from the resource's own finding history in DynamoDB.
- **Unknown changes are never guessed:** changes outside the knowledge base are labelled Review and escalated.
- **Failure isolation:** an error on one resource is logged and the scan continues with the others.
- **Least privilege:** collection permissions are read-only and scoped; remediation permissions exist only when `auto_remediate = true`.

Results

Classification performance

Model / split	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Random Forest – Split A (record-level, n_test=143)	1.000	1.000	1.000	1.000
Decision Tree – Split A	1.000	1.000	1.000	1.000
Random Forest – Split B (scenario held out, n_test=119)	0.370	0.267	0.333	0.296
Decision Tree – Split B	0.706	0.511	0.667	0.565

Interpretation. Split A meets the $\geq 95\%$ macro-F1 objective, but every scenario appears in both train and test, so it largely measures how well the model reproduces the labelling rules. Split B holds out entire scenarios (E1, E2, S3id, S5) and is far harder: two (resource type, risk) combinations never appear in training. We report both numbers deliberately; improving generalisation to unseen scenarios (more scenario templates, real drift data) is the main item of future work.

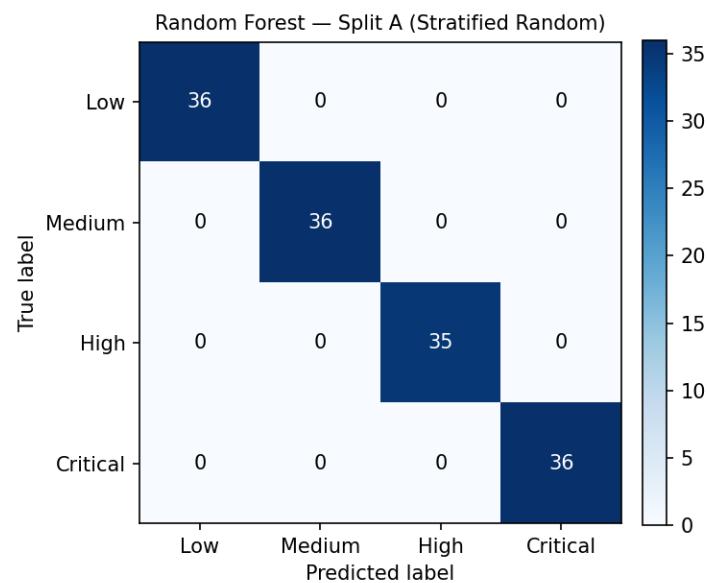


Figure 4. Confusion matrix, Random Forest, Split A.

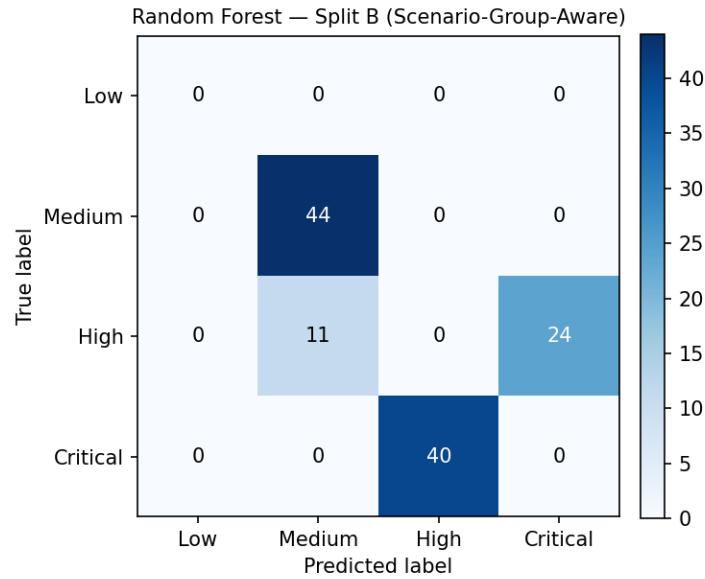


Figure 5. Confusion matrix, Random Forest, Split B (scenario held out).

Explainability (SHAP)

Rank	Feature	Mean SHAP
1	security_sensitivity	0.1247
2	change_magnitude	0.0886
3	drift_frequency	0.0332
4	resource_type_iam_policy	0.0299
5	changed_attribute_lifecycle_rule	0.0295
6	public_exposure	0.0272

SHAP additivity holds: the largest difference between base value + SHAP contributions and the model's predicted probability is 6.7e-16. The two most influential features are security_sensitivity and change_magnitude, matching the security reasoning the scenarios were designed around.

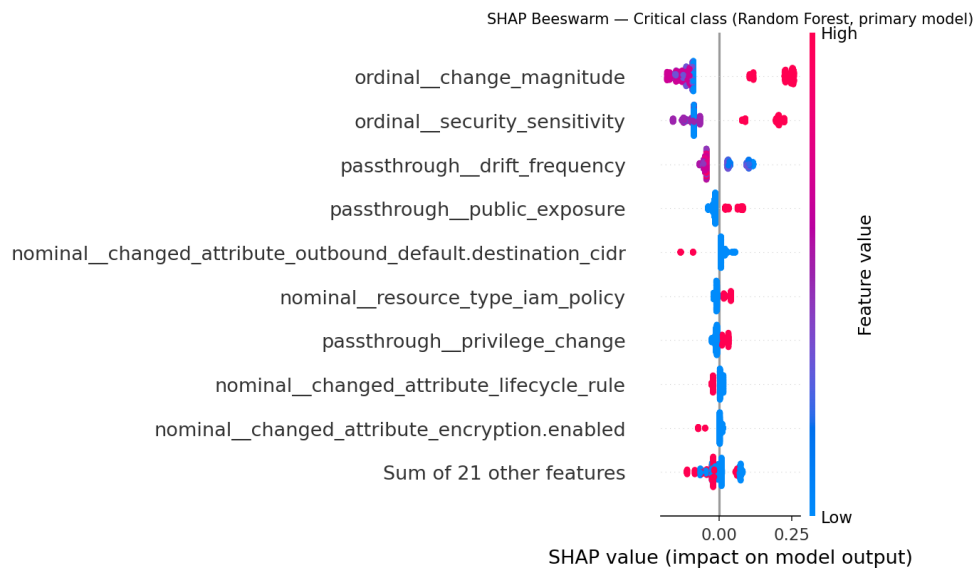


Figure 6. SHAP beeswarm for the Critical class.

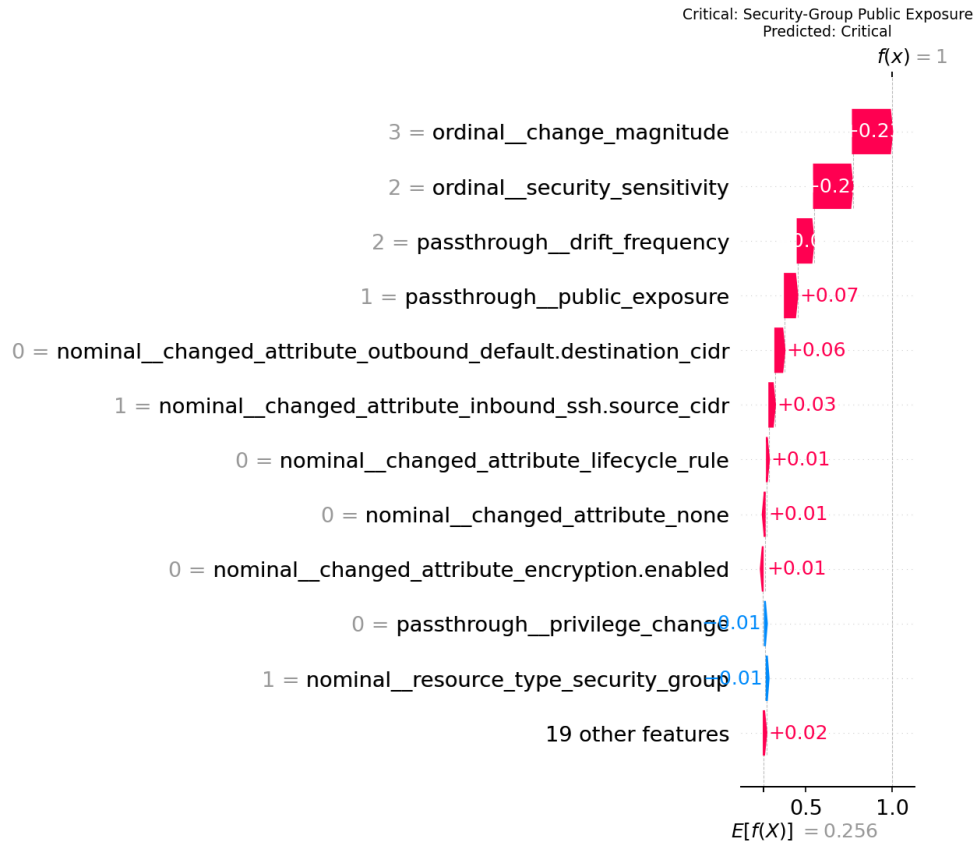


Figure 7. Local SHAP explanation of a Critical prediction (SSH opened to 0.0.0.0/0).

Cloud pipeline verification

The AWS pipeline was tested end-to-end against moto (an in-memory simulation of the AWS APIs), covering the exact resources Terraform deploys:

Test scenario	Expected	Result
Fresh deployment, no manual changes	No drift on all 3 resources, no alert	Pass
SSH (22) opened to 0.0.0.0/0	Critical, SNS alert, finding stored	Pass
S3 public-access block disabled + auto-remediate	Critical, block re-enabled, next scan clean	Pass
Public SSH + auto-remediate	Rule revoked, next scan clean	Pass
IAM inline policy escalated to Action:*/Resource:*	Critical	Pass
MFA condition removed from trust policy	High	Pass
Versioning enabled outside Terraform	Low/Medium, no alert	Pass
Same drift seen in 3 scans	drift_frequency = 0, 1, 2	Pass
HTTPS rule deleted (attribute never modelled)	Review + alert (not guessed)	Pass
Monitored bucket missing	Error recorded, other resources still scanned	Pass
API: list findings / history / unknown resource	200 / 200 / 404	Pass
Exported model vs scikit-learn on 711 records	Identical probabilities	Pass

Full suite: 218 tests passed, 1 skipped (the skipped test needs a real AWS account and runs with RUN_REAL_AWS_TESTS=1).

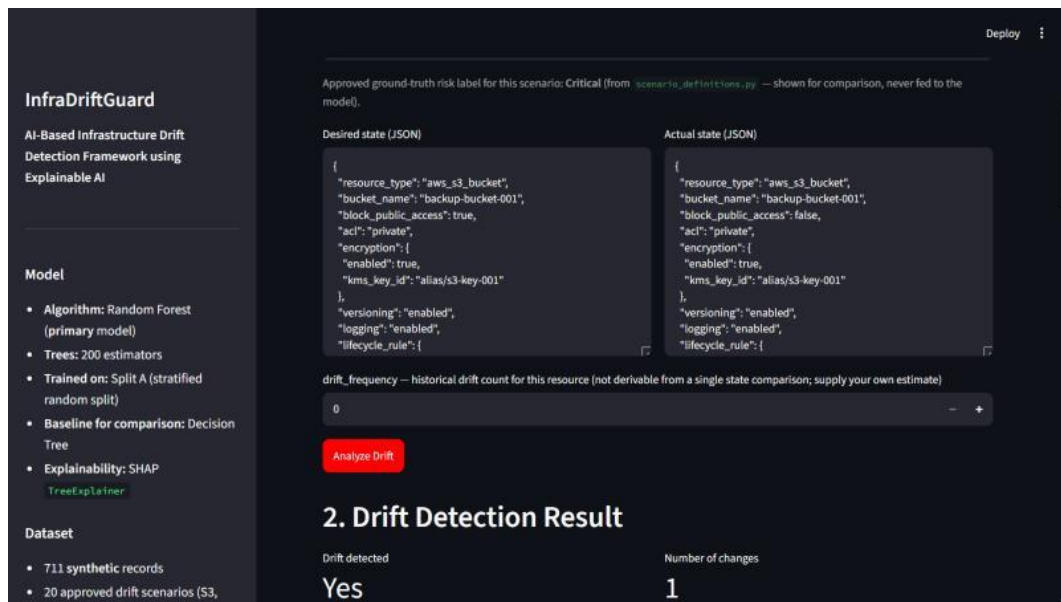


Figure 8. Streamlit dashboard – scenario S1 input.

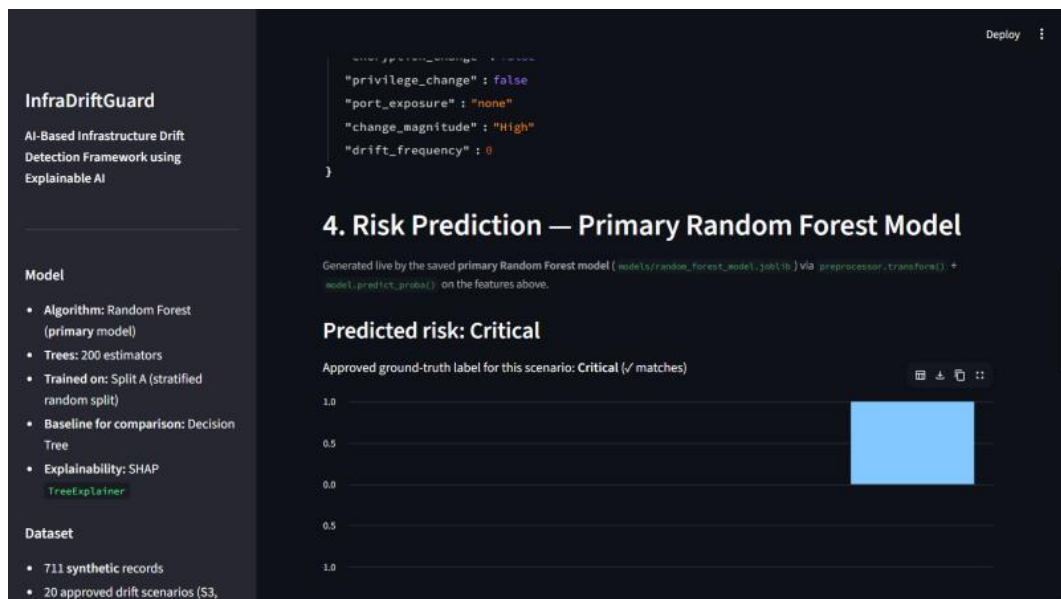


Figure 9. Dashboard prediction: Critical.

Deployment and Cost Control

7. Create an AWS account, enable MFA on the root user and create an IAM admin user for daily use.
8. Install Terraform ≥ 1.5 and the AWS CLI; run `aws configure`.
9. `cd src/aws/terraform && terraform init && terraform apply -var alert_email=<you@example.com>`
10. Confirm the SNS subscription e-mail; sign in to the dashboard with the temporary Cognito password.
11. `terraform output -json > outputs.json`; `streamlit run src/frontend/app.py` → Live AWS findings tab.
12. Demo drift: open SSH to 0.0.0.0/0 in the console, click Scan now, receive the Critical alert e-mail.
13. When finished: `terraform destroy` (removes everything, cost returns to \$0).

Expected monthly cost for the demo: US\$0.00 within free-tier limits (~720 Lambda invocations, a few hundred DynamoDB writes, a handful of e-mails). The AWS Budget alarm triggers at \$0.50 actual or \$1 forecast.

GitHub Repository and Contributions

Repository: `InfraDriftGuard_Cloud_Project_2026`

Branches: `main` ← `develop` ← `feature/student1`, `feature/student2`. Each student works only on their feature branch, opens pull requests into `develop`, reviews the partner's PRs, and `develop` is merged into `main` and tagged `v1.0-Phase1` once stable.

Activity	Student 1	Student 2
Literature survey	Papers 1–8	Papers 9–15
Research gap analysis	Papers 1–8	Papers 9–15
Dataset generation & validation	✓	
ML model training & evaluation audit		✓
SHAP explainability		✓
Drift engine & feature extraction (backend)	✓	
AWS collectors & Lambda handlers	✓	✓
Terraform infrastructure	✓	
Streamlit frontend & Cognito/API client		✓
Testing	✓	✓
Documentation & architecture diagrams	✓	✓
Presentation	✓	✓

Conclusion and Future Work

InfraDriftGuard closes the gap between static IaC analysis and runtime cloud security: it detects drift between Terraform and live AWS resources every hour, rates each drift with an explainable Random Forest, never guesses on unfamiliar changes, alerts within one scan cycle and can safely revert the most dangerous changes – all in a serverless design that costs nothing within the free tier.

Future work (Phase II):

- Collect real, anonymised drift events (e.g. CloudTrail-triggered scans) to train beyond synthetic data and improve the scenario-held-out score.
- Event-driven detection via CloudTrail + EventBridge rules for near-real-time response.
- Extend coverage to RDS, KMS keys, Lambda configuration and VPC network ACLs.
- Multi-account monitoring through AWS Organizations and cross-account read-only roles.
- Generate pull requests that update Terraform when a drift is intentional (accept vs revert workflow).

References

- [1] G. Thiagarajan, V. Bist and P. Nayak, "AI-Driven Configuration Drift Detection in Cloud Environments," *International Journal of Communication Networks and Information Security (IJCNIS)*, vol. 16, no. 5, pp. 721–743, 2024. (Scopus indexed)
- [2] M. M. Hassan, J. Salvador, S. K. Karmaker Santu and A. Rahman, "State Reconciliation Defects in Infrastructure as Code," *Proceedings of the ACM on Software Engineering (FSE)*, vol. 1, 2024.
- [3] S. Abuzakuk, L. Crijns, A.-M. Kermarrec, R. Pires and M. de Vos, "RIVA: Leveraging LLM Agents for Reliable Configuration Drift Detection," *ACM*, 2026 (arXiv:2603.02345).
- [4] J. Wen, Z. Chen, Z. Zhu, F. Sarro, Y. Liu, H. Ping and S. Wang, "LLM-Based Misconfiguration Detection for AWS Serverless Computing," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 35, no. 4, art. 110, 2026.
- [5] A. Rahman, S. I. Shamim, D. B. Bose and R. Pandita, "Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 32, no. 4, 2023.
- [6] N. Saavedra, J. Gonçalves, M. Henriques, J. F. Ferreira and A. Mendes, "Polyglot Code Smell Detection for Infrastructure as Code with GLITCH," *Proc. 38th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE), Tool Demonstrations*, 2023.
- [7] R. Opdebeeck, A. Zerouali and C. De Roover, "Control and Data Flow in Security Smell Detection for Infrastructure as Code: Is It Worth the Effort?," *Proc. 20th IEEE/ACM Int. Conf. on Mining Software Repositories (MSR)*, 2023.
- [8] A. Verdet, M. Hamdaqa, L. M. P. Da Silva and F. Khomh, "Assessing the adoption of security policies by developers in Terraform across different cloud providers," *Empirical Software Engineering (Springer)*, vol. 30, no. 3, 2025.
- [9] N. Saavedra, J. F. Ferreira and A. Mendes, "InfraFix: Technology-Agnostic Repair of Infrastructure as Code," *Proc. 34th ACM SIGSOFT Int. Symposium on Software Testing and Analysis (ISSTA), Tool Demonstrations*, 2025.
- [10] E. Low, B. Chen et al., "Repairing Infrastructure-as-Code using Large Language Models," *Proc. IEEE Secure Development Conference (SecDev)*, 2024.
- [11] M. Begoug, M. Chouchen, A. Ouni, E. A. AlOmar and M. W. Mkaouer, "Fine-Grained Just-In-Time Defect Prediction at the Block Level in Infrastructure-as-Code (IaC)," *Proc. 21st IEEE/ACM Int. Conf. on Mining Software Repositories (MSR)*, pp. 100–112, 2024.
- [12] M. Begoug, M. Chouchen and A. Ouni, "TerraMetrics: An Open Source Tool for Infrastructure-as-Code (IaC) Quality Metrics in Terraform," *Proc. 32nd IEEE/ACM Int. Conf. on Program Comprehension (ICPC), Tool Demonstrations*, 2024.
- [13] G. Rjoub et al., "A Survey on Explainable Artificial Intelligence for Cybersecurity," *IEEE Transactions on Network and Service Management*, vol. 20, no. 4, pp. 5115–5140, 2023.
- [14] V. Z. Mohale and I. C. Obagbuwa, "A systematic review on the integration of explainable artificial intelligence in intrusion detection systems to enhancing transparency and interpretability in cybersecurity," *Frontiers in Artificial Intelligence*, vol. 8, 2025, doi:10.3389/frai.2025.1526221.
- [15] S. M. Saleh, I. M. Sayem, N. Madhavji and J. Steinbacher, "Advancing Software Security and Reliability in Cloud Platforms through AI-based Anomaly Detection," *Proc. ACM Cloud Computing Security Workshop (CCSW)*, 2024.